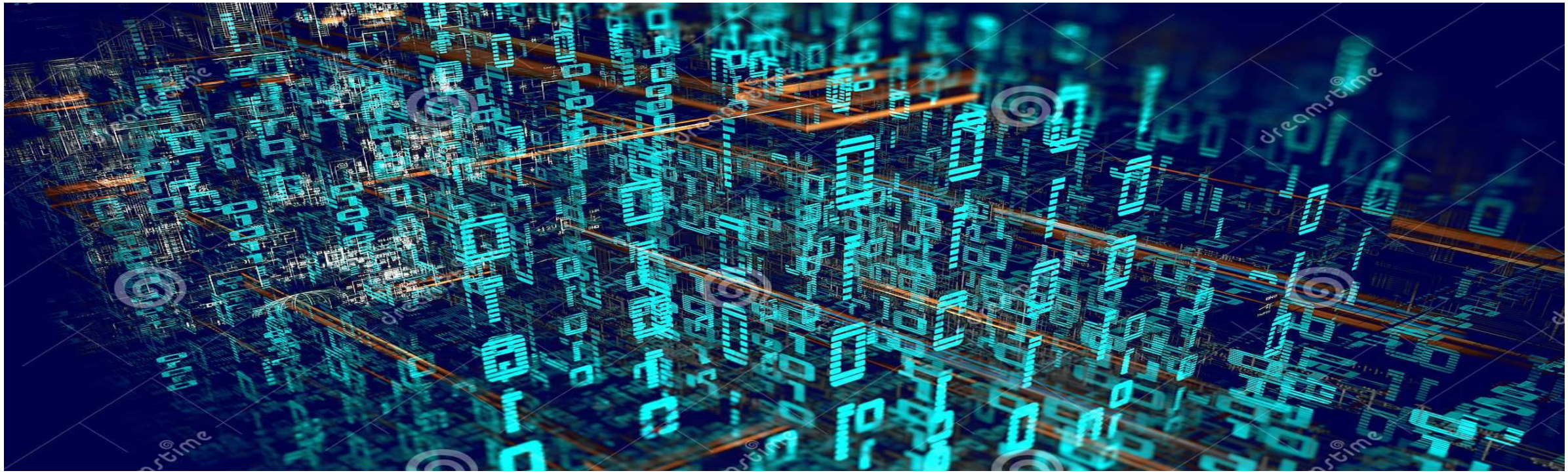
Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_COMPILER DESIGN



5

CODE OPTIMIZATION

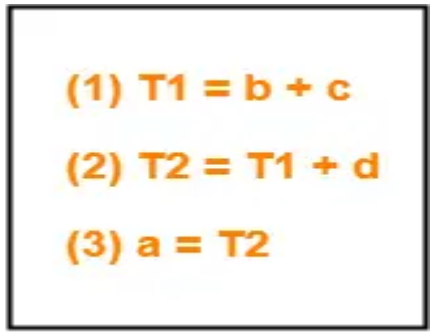
Dr. B.V. Baiju,
SCOPE,Assistant Professor
VIT, Vellore

Basic Blocks

- Basic Block is a set of statements that always ***executes one after other***, in a **sequence**.
- The first task is to ***partition a sequence of three-address code*** into basic blocks.
- A new basic block is begun with the first instruction and instructions are added until a jump or a label is met.
- In the absence of a jump, control moves further consecutively from one instruction to another.
- Three Address Code for the expression

a = b + c + d

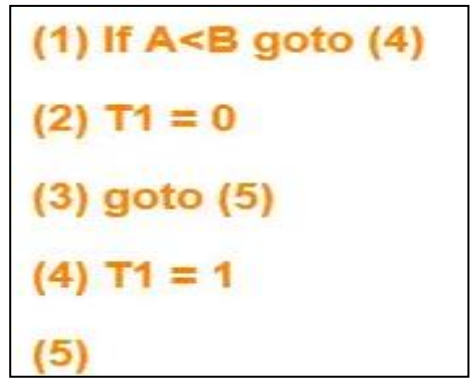
Basic Block



All the statements execute in a sequence one after the other.

If A<B then 1 else 0

Not A Basic Block



Statements do not execute in a sequence one after the other

Algorithm : Partitioning three-address instructions into basic blocks

INPUT: A ***sequence of three-address instructions***.

OUTPUT: A ***list of the basic blocks*** for that sequence in which each instruction is assigned to exactly one basic block.

METHOD:

- First, **identify the leaders** in the intermediate code, (i.e the first instructions in some basic block)
- The instruction just past the end of the intermediate program is not included as a leader.
- The **rules for finding leaders** are:
  1. The ***first three-address instruction*** in the intermediate code is a leader.
  2. Any instruction that is the ***target of a conditional or unconditional jump*** is a leader.
  3. Any instruction that ***immediately follows a conditional or unconditional jump*** is a leader.
- Each leader's basic block will have all the instructions from the leader itself until the instruction, which is just before the starting of the next leader.

Three Address Code

1. a = b + c
2. d = a - e
3. if (d < 0) goto L1
4. a = a * 2
5. L1: b = a + d
6. c = b - a

B1

1. a = b + c
2. d = a - e
3. if (d < 0) goto L1

B2

4. a = a * 2

B3

5. L1: b = a + d
6. c = b - a

Instruction 1 (a = b + c) is a leader (first instruction).

Instruction 3 (if (d < 0) goto L1) makes instruction 5 (L1: b = a + d) a leader.

Instruction 4 (a = a * 2) is a leader because it follows a conditional jump.

```
begin
  x := 10;
  y := 20;
  z := 0;

  if (x > y) then
    z := x + y;
  else
    z := x - y;

  result := z * 2;
end
```

Three Address Code

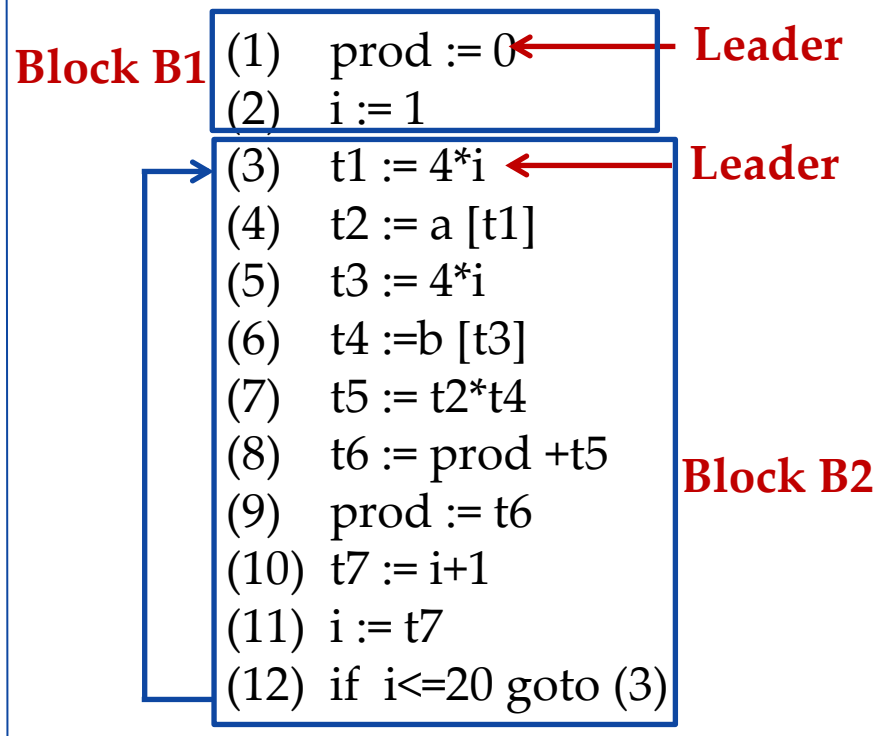
```
(1) x := 10
(2) y := 20
(3) t1 := x > y
(4) If false t1 goto (7)
(5) t2 := x + y
(6) z := t2
(7) goto (10)
(8) t3 := x - y
(9) z := t3
(10) t4 := z * 2
(11) result := t4
```

```
(1) x := 10    //Leader 1
(2) y := 20
(3) t1 := x > y
(4) If false t1 goto (7)
(5) t2 := x + y      // Leader 2
(6) z := t2
(7) goto (10)
(8) t3 := x – y    //Leader 3
(9) z := t3
(10) t4 := z * 2   //Leader 4
(11) result := t4
```

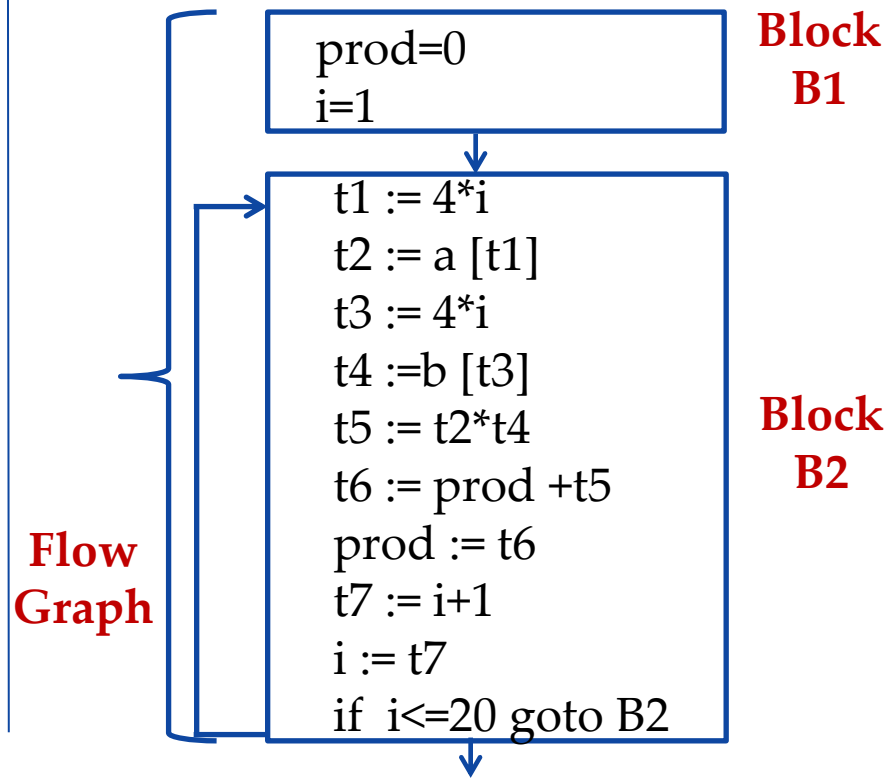
BB1: Instructions (1) to (4)
BB2: Instructions (5) to (7)
BB3: Instructions (8) to (9)
BB4: Instructions (10) to (11)

Example1: Partition into basic blocks

```
begin
     prod := 0;
     i := 1;
     do
prod := prod + a[t1] * b[t2];
      i := i+1;
     while  i<= 20
end
```



Three Address Code



Flow Graph

```
1. i = 1
2. while (i <= n) {
3.     j = 1
4.     while (j <= m) {
5.         k = i + j
6.         x = k * 2
7.         y = k + 3
8.         if (y > 10) {
9.             z = y * 2
10.        } else {
11.            z = y / 2
12.        }
13.        j = j + 1
14.    }
15.    i = i + 1
16.}
```

- **Example :** Consider the following source code for a 10 x 10 matrix to an identity matrix

```
for i from 1 to 10 do
        for j from 1 to 10 do
            a [ i, j ] = 0.0;
 for i from 1 to 10 do
            a [ i,i ] = 1.0;
```

```
1) i = 1                  //Leader 1 (First statement)
2) j = 1                  //Leader 2 (Target of 11th statement)
3) t1 = 10 * I            //Leader 3 (Target of 9th statement)
4) t2 = t1 + j
5) t3 = 8 * t2
6) t4 = t3 - 88
7) a[t4] = 0.0
8) j = j + 1
9) if j <= 10 goto (3)
10) i = i + 1             //Leader 4 (Immediately following Conditional goto statement)
11) if i <= 10 goto (2)
12) i = 1                 //Leader 5 (Immediately following Conditional goto statement)
13) t5 = i – 1            //Leader 6 (Target of 17th statement)
14) t6 = 88 * t5
15) a[t6] = 1.0
16) i = i + 1
17) if i <= 10 goto (13)
```

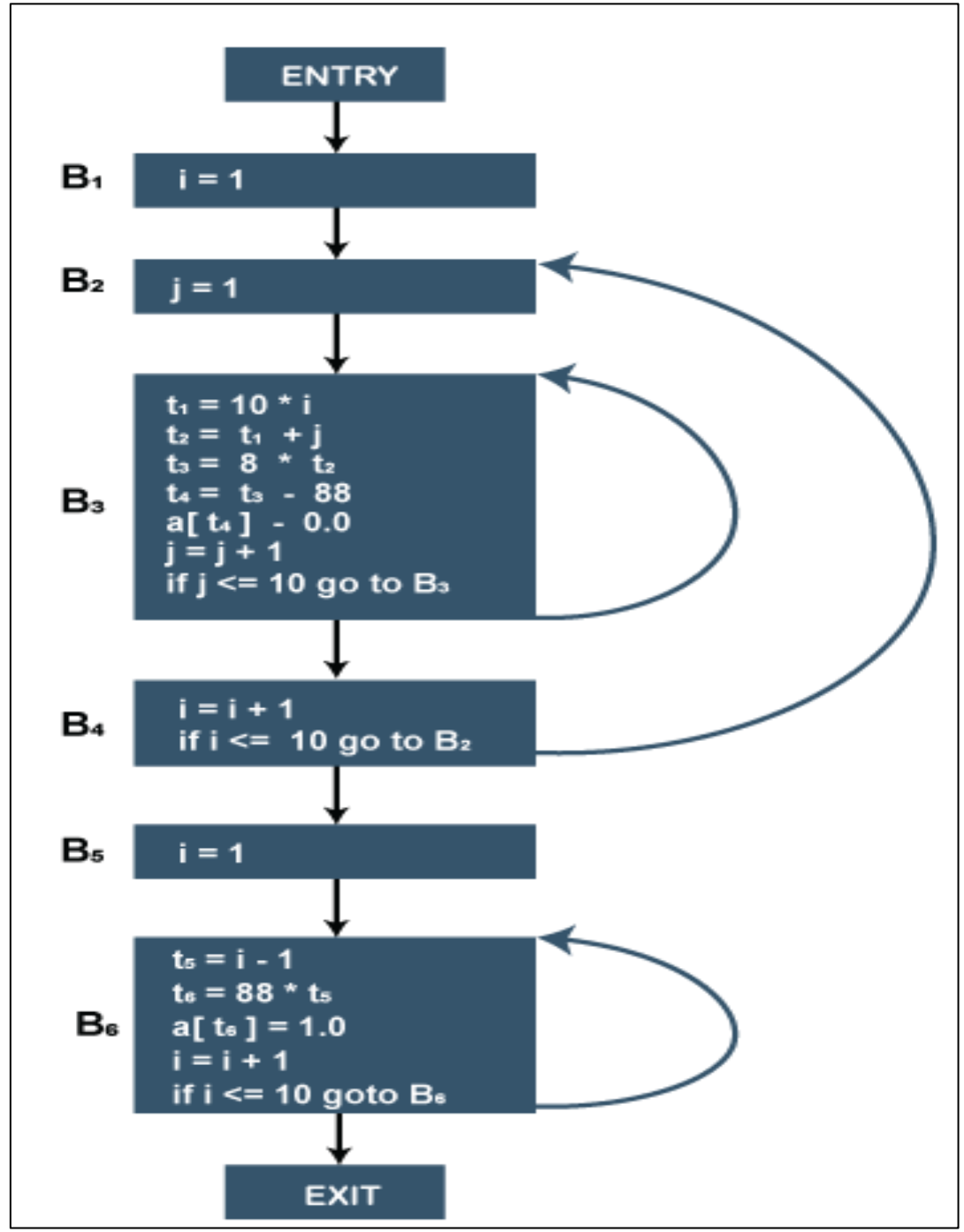
There are six basic blocks for the above-given code,

- B1 for statement 1
- B2 for statement 2
- B3 for statements 3-9
- B4 for statements 10-11
- B5 for statement 12
- B6 for statements 13-17.

Flow Graphs

- Flow graph is a directed graph containing the ***flow-of-control information*** for the set of basic blocks making up a program.
- The nodes/bocks of the control flow graph are the basic blocks of the program.
- A control flow graph shows how program control is parsed among the blocks.
- There are two designated blocks in Control Flow Graph:
  1. **Entry Block:** The entry block allows the control to enter in the control flow graph.
  2. **Exit Block:** Control flow leaves through the exit block.
- An edge can flow from one block X to another block Y in such a case when the Y block's first instruction immediately follows the X block's last instruction.
- The following ways will describe the edge:
  - There is a conditional or unconditional jump from the end of X to the starting of Y.
  - Y immediately follows X in the original order of the three-address code, and X does not end in an unconditional jump.

- Block B1 is the entry point for the flow graph because B1 contains starting instruction.
- B2 is the only successor of B1 because B1 doesn't end with unconditional jumps, and the B2 block's leader immediately follows the B1 block's leader.
- B3 block has two successors. One is a block B3 itself because the first instruction of the B3 block is the target for the conditional jump in the last instruction of block B3. Another successor is block B4 due to conditional jump at the end of B3 block.
- B6 block is the exit point of the flow graph.



```
 1)   i = 1
 2)   j = 1
 3)   t1 = 10 * i
 4)   t2 = t1 + j
 5)   t3 = 8 * t2
 6)   t4 = t3 - 88
 7)   a[t4] = 0.0
 8)   j = j + 1
 9)   if j <= 10 goto (3)
10)   i = i + 1
11)   if i <= 10 goto (2)
12)   i = 1
13)   t5 = i - 1
14)   t6 = 88 * t5
15)   a[t6] = 1.0
16)   i = i + 1
17)   if i <= 10 goto (13)
```